Deep Residual Learning for Image Recognition

Kaiming He, Xiangyu Zhang, Shaoqing Ren, Jian Sun (2015)

Reviewed by BeomSoo Son

Abstract

Problem

As CNNs became deeper, simply stacking more layers did not always improve performance; very deep plain networks became difficult to optimize.

ResNet proposal

Instead of directly learning the desired mapping H(x), each block learns the residual F(x) = H(x) − x, then adds the input back through a shortcut.

Key mechanism

Identity shortcut connections pass information forward without adding parameters, making extremely deep networks easier to optimize.

From Shallow CNNs to Very Deep CNNs

Classical CNN intuition

More layers can build richer low-, mid-, and high-level visual features.

Why depth matters

ImageNet-winning models before ResNet already used “very deep” architectures, such as VGG and GoogLeNet.

Problem

When the network going excessively deeper, an optimization problem occurred.

Degradation Problem

Not just vanishing gradients

Normalized initialization and Batch Normalization allow very deep nets to start converging.

Unexpected behavior

After a certain depth, accuracy saturates and then degrades rapidly.

Why it is surprising

The deeper model should be able to copy a shallower solution by using extra layers as identity mappings.

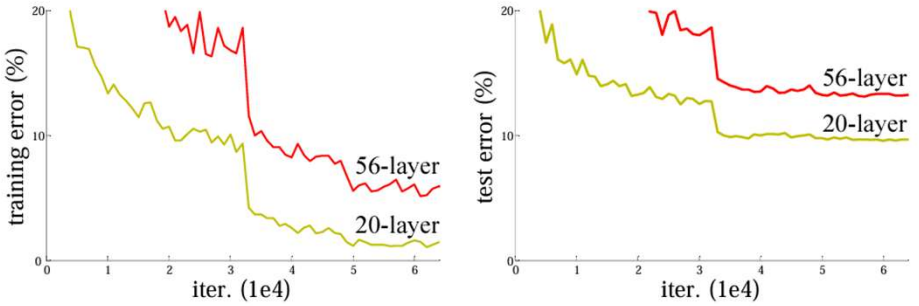


Figure 1. Training error (left) and test error (right) on CIFAR-10 with 20-layer and 56-layer "plain" networks. The deeper network has higher training error, and thus test error. Similar phenomena on ImageNet is presented in Fig. 4.

Residual Learning

Original goal

A stack of layers is expected to learn the desired mapping $H(x)$.

$$\text{Direct learning: } \boldsymbol{x} \rightarrow \boldsymbol{H(x)}$$

ResNet reformulation

Let the stacked layers learn only the residual function, then add the original input back.

$$\boldsymbol{F(x) = H(x) - x \Rightarrow H(x) = F(x) + x}$$

Residual Block

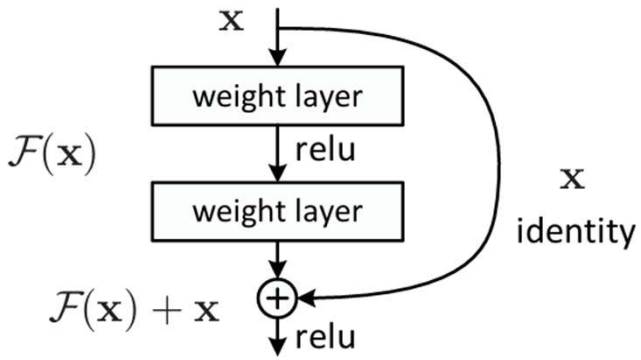


Figure 2. Residual learning: a building block.

Main path

A few weight layers learn the residual mapping F(x).

Shortcut path

The input x skips those layers and is added back to their output.

Output

The block outputs F(x) + x, followed by a ReLU activation.

Important property

An identity shortcut adds no new parameter and almost no computational cost.

Identity, Projection Shortcuts

Same input/output dimensions

$$\mathbf{y} = \mathcal{F}(\mathbf{x}, \{W_i\}) + \mathbf{x}.$$

Different dimensions

$$\mathbf{y} = \mathcal{F}(\mathbf{x}, \{W_i\}) + W_s\mathbf{x}.$$

Shortcut options tested

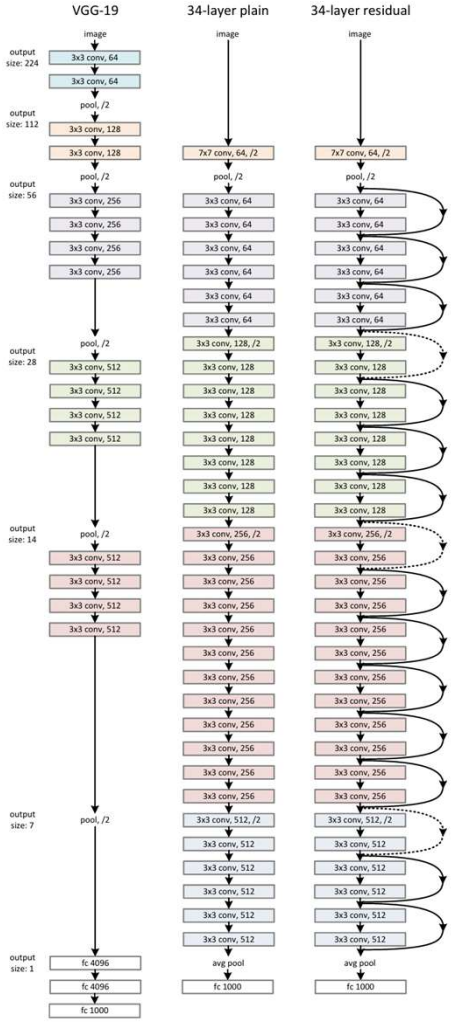
A: identity shortcut with zero-padding when dimensions increase
B: projection shortcut only when dimensions increase
C: projection shortcut for every shortcut

=> All residual versions outperform the plain network.

Key finding

Projection shortcuts improve slightly, but they are not essential for solving degradation.

Overall Architecture



Plain baseline

VGG-style design using mostly 3×3 convolutions.

Residual version

Add shortcut connections to the same 34-layer architecture.

Downsampling rule

When feature-map size halves, the number of filters doubles to preserve computation per layer.

Fair comparison

Plain and residual networks have the same depth, width, and most computational cost; only shortcut additions differ.

Bottleneck Design for Very Deep ResNets

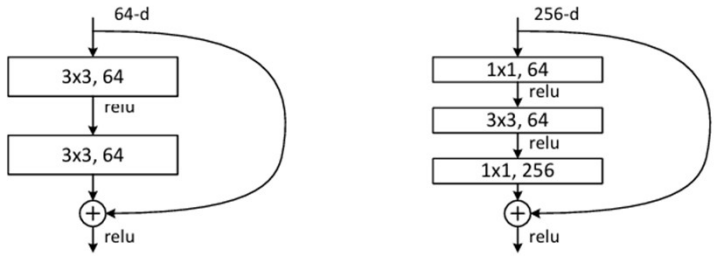


Figure 5. A deeper residual function $\mathcal{F}$ for ImageNet. Left: a building block (on 56×56 feature maps) as in Fig. 3 for ResNet-34. Right: a "bottleneck" building block for ResNet-50/101/152.

When excessively deeper network

ResNet replaces each 2-layer block with a 3-layer bottleneck block.

1×1 → 3×3 → 1×1

The first 1×1 conv reduces dimensions, the 3×3 conv processes the compact representation, and the last 1×1 conv restores dimensions.

Why it matters

It enables much deeper networks without making computation explode.

Training Setup

Dataset

ImageNet 2012 classification: 1.28M training images, 50k validation images, 1000 classes.

Data processing

Random scale augmentation, 224×224 random crop, horizontal flip, per-pixel mean subtraction, and standard color augmentation.

Optimization

SGD with mini-batch size 256, initial learning rate 0.1, momentum 0.9, and weight decay 0.0001.

Normalization and regularization

Batch Normalization is used after every convolution and before activation. The models are trained from scratch and do not use dropout.

Experiments: ImageNet Optimization

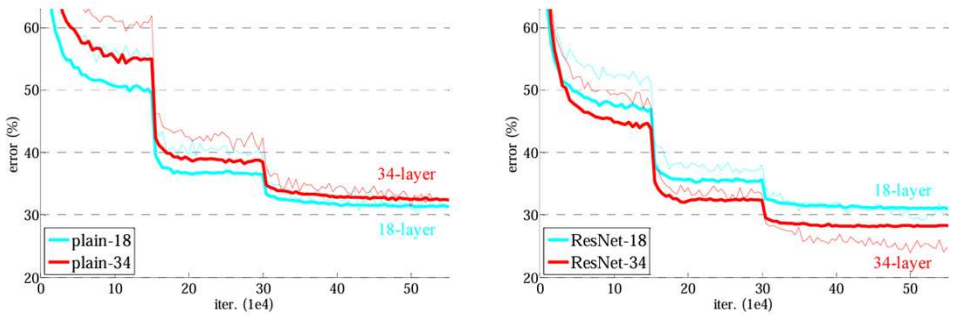


Figure 4. Training on **ImageNet**. Thin curves denote training error, and bold curves denote validation error of the center crops. Left: plain networks of 18 and 34 layers. Right: ResNets of 18 and 34 layers. In this plot, the residual networks have no extra parameter compared to their plain counterparts.

Plain networks

34-layer plain net shows higher training/validation error than 18-layer plain net.

Residual networks

With shortcuts, the 34-layer ResNet becomes better than the 18-layer ResNet.

Same parameter count

The key difference is the shortcut connection, not a larger model.

Experiments: ImageNet Accuracy

	plain	ResNet
18 layers	27.94	27.88
34 layers	28.54	**25.03**

Table 2. Top-1 error (%, 10-crop testing) on ImageNet validation. Here the ResNets have no extra parameter compared to their plain counterparts. Fig. 4 shows the training procedures.

model	top-1 err.	top-5 err.
VGG-16 [41]	28.07	9.33
GoogLeNet [44]	-	9.15
PReLU-net [13]	24.27	7.38
plain-34	28.54	10.02
ResNet-34 A	25.03	7.76
ResNet-34 B	24.52	7.46
ResNet-34 C	24.19	7.40
ResNet-50	22.85	6.71
ResNet-101	21.75	6.05
ResNet-152	**21.43**	**5.71**

Table 3. Error rates (%, **10-crop** testing) on ImageNet validation. VGG-16 is based on our test. ResNet-50/101/152 are of option B that only uses projections for increasing dimensions.

method	top-1 err.	top-5 err.
VGG [41] (ILSVRC'14)	-	8.43†
GoogLeNet [44] (ILSVRC'14)	-	7.89
VGG [41] (v5)	24.4	7.1
PReLU-net [13]	21.59	5.71
BN-inception [16]	21.99	5.81
ResNet-34 B	21.84	5.71
ResNet-34 C	21.53	5.60
ResNet-50	20.74	5.25
ResNet-101	19.87	4.60
ResNet-152	**19.38**	**4.49**

Table 4. Error rates (%) of **single-model** results on the ImageNet validation set (except † reported on the test set).

method	top-5 err. (**test**)
VGG [41] (ILSVRC'14)	7.32
GoogLeNet [44] (ILSVRC'14)	6.66
VGG [41] (v5)	6.8
PReLU-net [13]	4.94
BN-inception [16]	4.82
ResNet (ILSVRC'15)	**3.57**

Table 5. Error rates (%) of **ensembles**. The top-5 error is on the test set of ImageNet and reported by the test server.

Experiments: CIFAR-10 and Extreme Depth

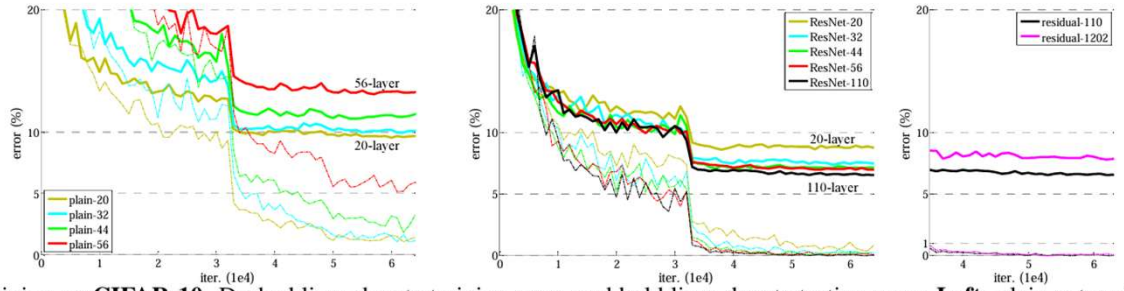


Figure 6. Training on **CIFAR-10**. Dashed lines denote training error, and bold lines denote testing error. **Left**: plain networks. The error of plain-110 is higher than 60% and not displayed. **Middle**: ResNets. **Right**: ResNets with 110 and 1202 layers.

Interpretation

Plain networks again degrade as depth increases, while ResNets train successfully and improve up to 110 layers. The 1202-layer model trains, but likely overfits the small CIFAR-10 dataset.

Detection and Segmentation

training data	07+12	07++12
test data	VOC 07 test	VOC 12 test
VGG-16	73.2	70.4
ResNet-101	**76.4**	**73.8**

Table 7. Object detection mAP (%) on the PASCAL VOC 2007/2012 test sets using **baseline** Faster R-CNN. See also Table 10 and 11 for better results.

metric	mAP@.5	mAP@[.5, .95]
VGG-16	41.5	21.2
ResNet-101	**48.4**	**27.2**

Table 8. Object detection mAP (%) on the COCO validation set using **baseline** Faster R-CNN. See also Table 9 for better results.

Key point

The detection system stays the same; the improvement comes from stronger learned representations.

COCO improvement

The COCO standard metric improves by +6.0 mAP points, which the paper reports as a 28% relative improvement.

Competition impact

ResNet-based submissions won 1st place in ImageNet detection, ImageNet localization, COCO detection, and COCO segmentation.

Conclusion

Main achievement

ResNet was proposed to solve the degradation problem, where training error increases as networks become deeper.

Why it works

Instead of directly learning the full mapping $H(x)$, ResNet learns the residual $F(x)$ and constructs $H(x) = F(x) + x$ through shortcut connections.

Why the architecture matters

This makes identity mapping easier to learn, since the network only needs to make $F(x)$ close to zero when little or no transformation is required.

Efficient deep networks

Bottleneck structures make very deep networks such as ResNet-50, 101, and 152 computationally efficient, allowing increased depth to lead to better performance.

Thank You